

Strings

08 July 2026 20:26

String is a sequence which is made up of one or more Unicode characters. Here the character can be a letter, digit, white space or any other symbol. A string can be created by enclosing one or more characters in a single, double, or triple quote.

```
print('\u20b9')
₹
print('\u0048')
H
print('\u0040')
@
print('\u0041')
A
```

Unicode values, they are 4 digit hexadecimal values.

```
str1 = 'Hello! python'
str2 = "Shiv Ray"
str3 = """Hello shiv
```

← Triple quote is used for multiline string.

```
How r u?
Are you coding!
```

characters inserted by Python when we use triple quote.

```
"""
str3
'Hello shiv\nHow r u?\nAre you coding!\n'
print(str3)
Hello shiv
How r u?
Are you coding!
```

```
str1
'Hello! python'
str2
'Shiv Ray'
str3
```

Internally Python holds the string into single quotes.

```
str4 = '''I am coding
in Python
not in Java'''
str4
'I am coding\nin Python\nnot in Java'
print(str4)
I am coding
in Python
not in Java
```

0	1	2	3	4	5	6	7	8	9	10	11	12
H	e	l	l	o	!		p	y	t	h	o	n
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

Accessing characters in a string

Each individual character in a string can be accessed using a technique called indexing. The index specifies the character to be accessed in the string and is written in square brackets([]). The index of the first character (from left) in the string is zero and the last character is N minus 1. Where N is the length of the string. If we give index value out of this range then we get an index error. The index must be an integer

(positive, 0 or negative).

'0' index represents the first character of the string and '-1' index represents the last character of the string.

For example:

```
str1
'Hello! python'
str1[0]
'H'
str1[1]
'e'
str1[2]
'l' ← first character
str1[-1]
'n'
```

```
len(str1)
```

```
13
```

```
str1[20] ← last character.
```

```
Traceback (most recent call last):
```

```
File "<pyshell#26>", line 1, in <module>
```

```
← str1[20] — total number of characters in the string.
```

```
IndexError: string index out of range
```

```
str1[-20]
```

```
Traceback (most recent call last):
```

```
File "<pyshell#27>", line 1, in <module>
```

```
str1[-20]
```

```
IndexError: string index out of range
```

Python allows an index value to be negative also. Negative indices are used when we want to access the character of the string from right to left.

Starting from right hand side, the first character has the index as -1 and the last character has the index -N. Where N is the length of the string.

Accessing the first character and the last character in another way.

```
n = len(str1)
n
13
str1[n-1]
'n'
str1[-n]
'H'
str1
'Hello! python'
```

0	1	2	3	4	5	6	7	8	9	10	11	12
H	e	l	l	o	!			p	y	t	h	n
-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

String is Immutable

A string is an immutable data type. It means that the content of the string cannot be changed after it has been created. An attempt to do this would lead to an error.

```
str1
'Hello! python'
str1[0] = 'h'
Traceback (most recent call last):
```

```
File "<pyshell#35>", line 1, in <module>
    str1[0] = 'h'
TypeError: 'str' object does not support item assignment
```

String Operations

As we know that String is a sequence of characters. Python allows certain operations on string data types, such as concatenation, repetition, membership, and slicing.

Concatenation: To concatenate means to join. Python allows us to join two strings using concatenation operator which is denoted by symbol +.

Example:

```
fname = 'Shiv'
lname = 'Ray'
fullName = fname + ' ' + lname
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
```

Repetition

Python allows us to repeat the given string using repetition operator which is denoted by symbol *.

For example:

```
fname
'Shiv'
lname
'Ray'
fullName
'Shiv Ray'
char = '%'
newchars = char * 10
newchars
'%%%%%%%%%'
lname * 5
'RayRayRayRayRay'
6 * fname
'ShivShivShivShivShivShiv'
char
'%'
fname
'Shiv'
```

For repetition:

one operand must be of type 'str'
and other operand must be of type 'int'.

Membership: Python has two membership operators, 'in' and 'not in'. The

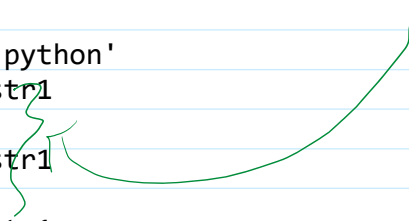
'in' operator takes two strings and returns True if the first string appears as a substring in the 2nd string, otherwise it returns False.

For example:

```
str1
'Hello! python'
'o' in str1
```

It will not change the original variable. The data will remain as it is.

```
str1
'Hello! python'
'o' in str1
True
'y' in str1
True
'P' in str1
False
'Py' in str1
False
'py' in str1
True
```



The '**not in**' operator also takes two strings and returns True if the 1st string does not appear as the substring in the 2nd string. Otherwise returns False.

For example:

```
str1
'Hello! python'
'x' not in str1
True
'p' not in str1
False
'py' not in str1
False
'pyc' not in str1
True
```

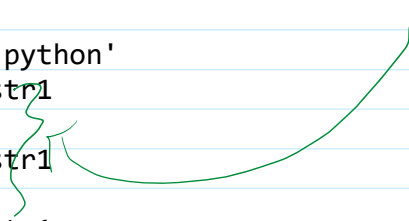
Slicing

In Python to access some part of string or substring, we use a method called slicing. This can be done by specifying an index range. Given a string STR one. The slice operation STR1 [N:M] Returns the part of the string STR1 starting from index N (Inclusive) And ending at M (exclusive). In other words, we can say that STR [N: M] Returns all the characters starting from STR1 [N] till STR [M-1]. The number of characters in a substring will always be equal to the difference of two indices M and N i.e., (M - N)

```
str1 = 'Computer Application'
```

For example:

```
str1 = 'Computer Application'
substr1 = str1[9:20]
substr1
'Application'
substr2 = str1[0:9]
substr2
'Computer'
```



0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
C	o	m	p	u	t	e	r		A	p	p	l	i	c	a	t	i	o	n

'Computer '

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19
C	O	m	p	u	t	e	r		A	p	p	L	i	c	a	t	i	O	n

Str_var[start:end:skip]

-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
-----	-----	-----	-----	-----	-----	-----	-----	-----	----	----	----	----	----	----	----	----	----

substr3

'Application'

substr3[len(substr3)-1:-1]

''

substr3[len(substr3)-1:0]

''

optional
optional
optional
is compulsory

This is required when 3rd option is given.

Reversing the string:

substr3[len(substr3)-1::-1]

'noitacilppA'

Or

substr3[::-1]

'noitacilppA'

} We want to do reverse, but we got empty string because the default movement is forward direction. Therefore we need to change the direction by using 3rd parameter. If we put '-' in the 3rd parameter then direction will be reversed.

substr3

'Application'

substr3[:5]

'Appli'

substr3[5:8]

'cat'

substr3[::-2]

'Apiain'

substr3[::-2]

'niaipA'

Traversing a String

We can access each character of a string or traverse a string using for loop and while loop.

String traversal using for loop:

```
for c in substr3:
```

```
    print(c)
```

A

p

p

l

i
c
a
t
i
o
n

String traversal using while loop

For example:

```
index = 0
while index < len(substr3):
    print(substr3[index])
    index += 1
```

A
p
p
l
i
c
a
t
i
o
n

String methods and built in functions

Python has several built in functions that allow us to work with strings.

Method	Description	Example
len()	Returns the length of the given string.	<pre>substr3 'Application' len(substr3) 11</pre>
title()	Returns the string with the first letter of every word in the string in uppercase and rest in lowercase.	<pre>str2 = 'hi shiv!, how are you?' tstr2 = str2.title() tstr2 'Hi Shiv!, How Are You?'</pre>
lower()	Returns the string with all uppercase letters converted to lowercase, and lowercase will remain at lowercase.	<pre>tstr2 'Hi Shiv!, How Are You?' ltstr2 = tstr2.lower() ltstr2 'hi shiv!, how are you?'</pre>
upper()	He returns the string with all lowercase letters converted to uppercase. And uppercase will remain at upper case.	<pre>tstr2 'Hi Shiv!, How Are You?' uptstr2 = tstr2.upper() uptstr2 'HI SHIV!, HOW ARE YOU?'</pre>

<code>count(str, start, end)</code>	It returns number of times substring STR occurs in the given string. If we do not give the start index and end index then searching starts from index 0 and ends at length of the string.	<pre> tstr2 'Hi Shiv!, How Are You?' count0 = tstr2.count('o') count0 2 count0 = tstr2.count('i') count0 2 count0 = tstr2.count('b') count0 0 count0 = tstr2.count('o', -5, -1) count0 1 </pre>
<code>find(str, start, end)</code>	Returns the first occurrence of index of substring str occurring in the given string. If we do not give start and end then searching starts from index 0 and ends at the length of the string. If the substring is not present in the given string, then the function returns -1	<pre> str1 = 'Hello World! Hello Hello' str1.find('Hello') 0 str1.find('llo') 2 str1.find('llo', 12, 20) 15 str1.find('llo', 14, 20) 15 str1.find('llo', 15, 20) 15 str1.find('llo', 16, 20) -1 str1.find('llo', 16) 21 </pre> ← 'llo' doesn't exists in the range.
<code>index(str, start, end)</code>	Same as find() but raises an exception if the substring is not present in the given string.	<pre> str1 = 'Hello World! Hello Hello' str1.index('o', 7) 7 str1.index('o', 8) 17 str1.index('p', 8) Traceback (most recent call last): File "<pyshell#10>", line 1, in <module> str1.index('p', 8) ValueError: substring not found </pre>
<code>endswith()</code>	Returns True if the given string ends with the supplied substring, otherwise returns False.	<pre> str1 'Hello World! Hello Hello' str1.endswith('Hello') True str1.endswith('Hello!') False </pre>
<code>startswith()</code>	The returns True is the	<pre> str1 </pre>

	given string starts with the supplied substring, otherwise returns False.	'Hello World! Hello Hello' str1.startswith('Hello') True str1.startswith('Hello!') False
isalnum()	Returns True if the characters of the given string are either alphabets or numeric. If white space or special symbols are part of the given string or the string is empty, it returns False.	username = 'shiv_123' username.isalnum() False username = 'shiv 123' username.isalnum() False username = 'shiv123' username.isalnum() True
islower()	Returns True if the string is non-empty and has lowercase alphabets or has at least one character as lowercase alphabet and rest are non-alphabet characters.	username 'shiv123' username.islower() True username = 'Shiv123' username.islower() False username = 's123' username.islower() True username = '1s23' username.islower() True username = '1s23S' username.islower() False
isupper()	Returns True if the string is non-empty and has all uppercase alphabets, or has at least one character as uppercase character and rest are non-alphabet characters.	username '1s23S' username.isupper() False username = '123S' username.isupper() True username = 'SHIV123' username.isupper() True
isspace()	Returns True if the string is non empty and all characters are white spaces. When (blank, tab newline, carriage return)	str1 = ' \n \t \r' str1.isspace() True str1 = 'Hello \n World!' str1.isspace() False
istitle()	Returns True if the string is non empty and title case i.e., the first letter of every word in the string in	str1 'Hello \n World!' str1.istitle() True str2 = 'This is code'

	upper case and the rest in lower case.	<code>str2.istitle()</code> <code>False</code>
<code>lstrip()</code>	Returns the string after removing the spaces only on the left of the string.	<code>str1 = ' Shiv'</code> <code>str1.lstrip()</code> <code>'Shiv'</code>
<code>rstrip()</code>	Returns the string after removing the spaces only on the right of the string.	<code>str2 = 'Shiv '</code> <code>str2.rstrip()</code> <code>'Shiv'</code>
<code>strip()</code>	Returns the string after removing the spaces both on the left and the right of the string.	<code>str1 = ' Shiv '</code> <code>str1.strip()</code> <code>'Shiv'</code>
<code>replace(oldstr, newstr)</code>	Replaces all occurrences of old string with the new string.	<code>str1 = 'Hello World!'</code> <code>str1.replace('o', '*')</code> <code>'Hell* W*rld!'</code>
<code>join()</code>	Returns a string in which the characters in the string have been joined by a separator.	<code>str1</code> <code>'Hello World!'</code> <code>str2 = '-'</code> <code>str2.join(str1)</code> <code>'H-e-l-l-o- -W-o-r-l-d-!'</code> <code>sep = '#Arjun#'</code> <code>sep.join(str1)</code> <code>'H#Arjun#e#Arjun#l#Arjun#l#Arjun#o#Arjun#</code> <code>#Arjun#W#Arjun#o#Arjun#r#Arjun#l#Arjun#d#Arjun#!'</code>
<code>partition()</code>	Partitions the given string at the first occurrence of the substring(separator) and returns the string partitioned into three parts. 1.Substring before the separator 2.Separator 3.Substring after the separator. If the separator is not found in the string, it returns the whole string itself and two empty strings	<code>str1 = 'india is a great country'</code> <code>str1.partition('is')</code> <code>('india ', 'is', ' a great country')</code> <code>str1.partition('was')</code> <code>('india is a great country', '', '')</code>
<code>split()</code>	Returns a list of words delimited by the specified substring. If no delimiter is given then words are separated	<code>str1</code> <code>'india is a great country'</code> <code>str1.split()</code> <code>['india', 'is', 'a',</code>

	by space.	'great', 'country'] str1 = 'indiasissasgreatscountry' str1.split('s') ['india', 'i', '', 'a', 'great', 'country']
--	-----------	--

Homework:

1. Write a program to input line(s) of text from the user until enter is pressed. Count the total number of characters in the text (including white spaces), total number of alphabets, total number of digits, total number of special symbols and total number of words in the given text. (Assume that each word is separated by one space).
2. Write a user defined function to convert a string with more than one word into title case string where string is passed as parameter. (Title case means that the first letter of each word is capitalised)
3. Write a function deleteChar() which takes two parameters one is a string and other is a character. The function should create a new string after deleting all occurrences of the character from the string and return the new string.
4. Input a string having some digits. Write a function to return the sum of digits present in this string.
5. Write a function that takes a sentence as an input parameter where each word in the sentence is separated by a space. The function should replace each blank with a hyphen and then return the modified sentence

Handling Strings

Write a program with a user defined function to count the number of times a character (passed as argument) occurs in a given string.

```
# Write a program with a user defined function to count the number of
times a character
# (passed as argument) occurs in a given string.
def charCount(ch, st):
    count = 0
    for character in st:
        if character == ch:
            count += 1
    return count
# main code
sent = input('Enter any sentence: ')
char = input('Enter any character to be searched: ')
count = charCount(char, sent)
print(f"Number of times character '{char}' occurs in the string:
```

```
{count}")
c = sent.count(char)
print(f"Number of times character '{char}' occurs in the string: {c}")
```

Output:

```
Enter any sentence: Shiv is learning devopps
Enter any character to be searched: i
Number of times character 'i' occurs in the string: 3
Number of times character 'i' occurs in the string: 3
```

Write a function to replace all vowels in the string with '*'

```
# Write a function to replace all vowels in the string with '*'
def replaceVowelWithChar(st,ch='*'):
    newstr = '' # empty string
    for character in st:
        if character in 'aeiouAEIOU':
            newstr += ch
        else:
            newstr += character
    return newstr
# main code
st = input('Enter any sentence: ')
nst = replaceVowelWithChar(st)
print(nst)
```

Output:

```
Enter any sentence: Shiv is learning the coding
Sh*v *s l**rn*ng th* c*d*ng
```

Write a program to input a sentence and reverse each word.

```
Input: Shiv is learning how to code
Output: vihS si gninrael who ot edoc
```

Write a program using a user defined function to check if a string is a palindrome or not. (A string is called a palindrome if it reads some backward as forward. For example, Kanak is a palindrome.)

```
def checkPalindrome(st):
    sc = st.lower()
    return sc[::-1] == sc
# main code
word = input('Enter any word: ')
print('Palindrome' if checkPalindrome(word) else 'Not Palindrome')
```

Output:

```
Enter any word: Mom
Palindrome
PS C:\Users\bluej\Documents\Students\ShivRay\ForG
ShivRay/ForGit/Coding/StringPrograms/four.py
Enter any word: Nitin
Palindrome
PS C:\Users\bluej\Documents\Students\ShivRay\ForG
ShivRay/ForGit/Coding/StringPrograms/four.py
Enter any word: Arora
Palindrome
PS C:\Users\bluej\Documents\Students\ShivRay\ForG
ShivRay/ForGit/Coding/StringPrograms/four.py
Enter any word: shiv
Not Palindrome
```